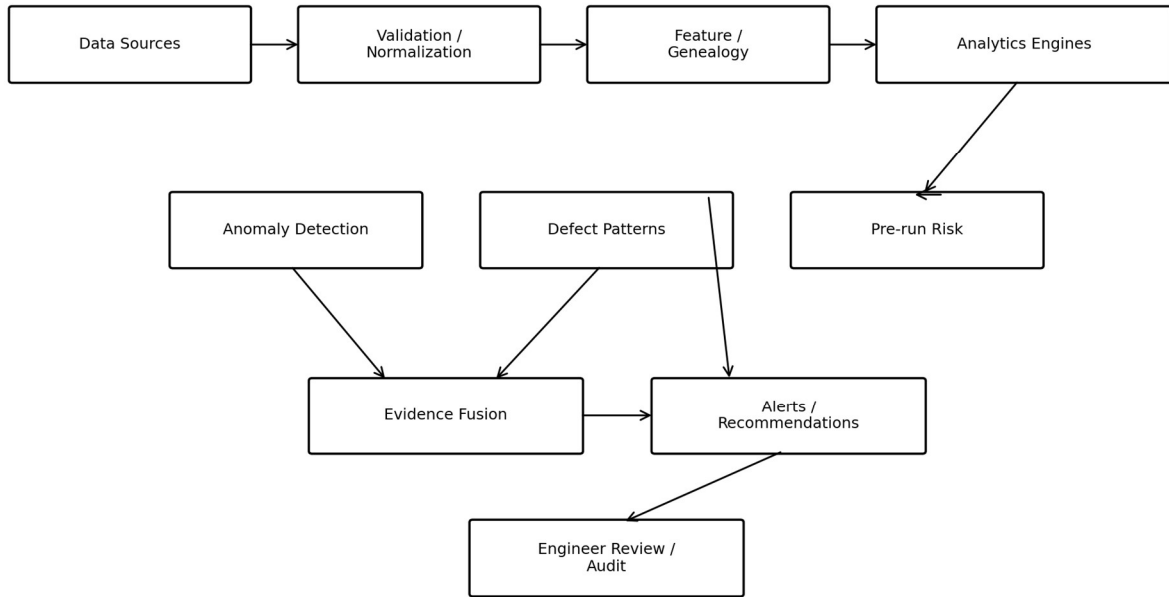


TEAM LOGICX

S1 — Wafer Yield Root Cause & Defect Pattern Analyser

Technical Documentation • Industry Context • Architecture • Algorithms • Demo & Judging Guide

LogicX: End-to-End Analytics Architecture



IBM Bobathon | Prototype Documentation

1. Executive Summary

LogicX is a semiconductor manufacturing analytics control-tower prototype designed to help process and yield engineers investigate wafer yield loss, identify spatial defect patterns, rank evidence-supported root-cause candidates, recommend engineer-reviewed actions, and flag upcoming lots with elevated pre-run risk.

The system is intentionally designed as a structured software platform rather than a conversational AI chatbot. Its critical analytical path uses deterministic and statistical services, traceable evidence identifiers, timestamps, algorithm versions, and human approval gates.

Positioning statement

LogicX is a decision-support prototype for yield engineering. It does not directly control fab equipment, does not claim causal certainty, and must be validated with representative fab data before any production deployment.

2. Problem Statement

At advanced semiconductor process nodes such as 3nm and 5nm, even a small yield reduction can create significant economic impact. Manufacturing information is distributed across equipment sensors, process recipes, lot and wafer genealogy, maintenance logs, metrology, inspection data, and defect maps. Engineers may need to correlate these sources manually to determine what changed, when it changed, and which equipment or process conditions are associated with the outcome.

The challenge has two connected requirements:

- Post-event investigation: explain yield or defect excursions using evidence from multiple manufacturing data sources.
- Pre-run prediction: identify upcoming lots that may be at elevated risk using only information available before processing starts.

The central technical difficulty is that correlation is not causation. Therefore, the system must expose evidence and uncertainty rather than presenting an unverified AI-generated explanation as fact.

3. Objectives

- Unify manufacturing entities through lot, wafer, tool, chamber, recipe, run, timestamp, metrology, inspection, maintenance, and yield relationships.
- Detect abnormal process behavior using baseline and multivariate analytical techniques.
- Classify spatial wafer defect distributions into interpretable pattern categories.
- Compare affected lots with historically similar lots and equipment conditions.
- Generate a deterministic evidence-fusion ranking of candidate root causes.
- Score upcoming lots using a strictly pre-run-safe feature set.
- Provide recommendations and alerts that require engineer review.
- Preserve traceability through evidence IDs, timestamps, and algorithm versions.
- Demonstrate reproducible synthetic scenarios without exposing proprietary fab data.

4. Industry Context

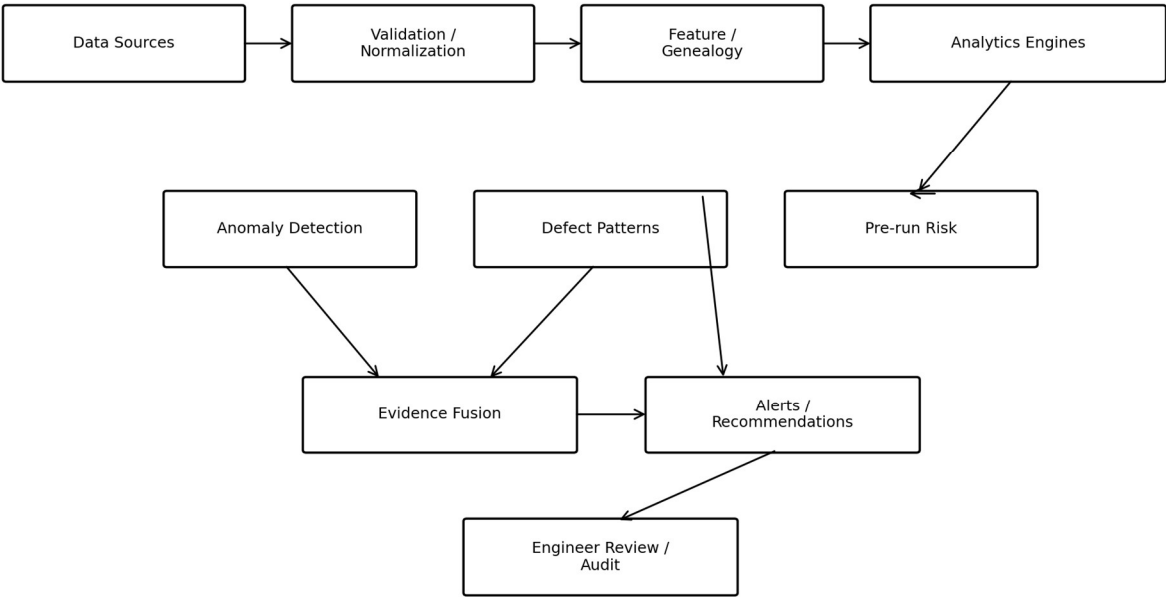
A semiconductor fab is a highly controlled manufacturing environment. A product wafer passes through many process modules, and each module may involve multiple tools, chambers, recipes, and operating conditions. The same wafer may generate large volumes of sensor, trace, metrology, and inspection records.

Typical information categories include:

Data category	Examples	Why it matters
Genealogy	Lot ID, wafer ID, parent/child relationships	Connects a result to its manufacturing history
Equipment	Tool, chamber, module, maintenance state	Supports recurrence and equipment-level investigation
Recipe/process	Recipe version, pressure, temperature, RF power, gas flow	Describes operating conditions
Trace data	Time series, alarms, process duration	Detects drift and transient excursions
Metrology	Overlay, critical dimension, film thickness	Provides measured process quality signals
Inspection	Defect type, x/y location, severity	Supports defect classification and spatial analysis
Yield	Good/bad die, wafer result, binning	Defines the outcome being investigated

5. Solution Overview

LogicX: End-to-End Analytics Architecture



The intended pipeline is:

1. Ingest → Validate → Normalize
2. Build time-aware genealogy and features
3. Run anomaly, defect-pattern, similarity, and pre-run risk analytics
4. Fuse evidence into candidate root-cause rankings
5. Generate alerts and engineer-reviewed recommendations
6. Record decisions and audit information

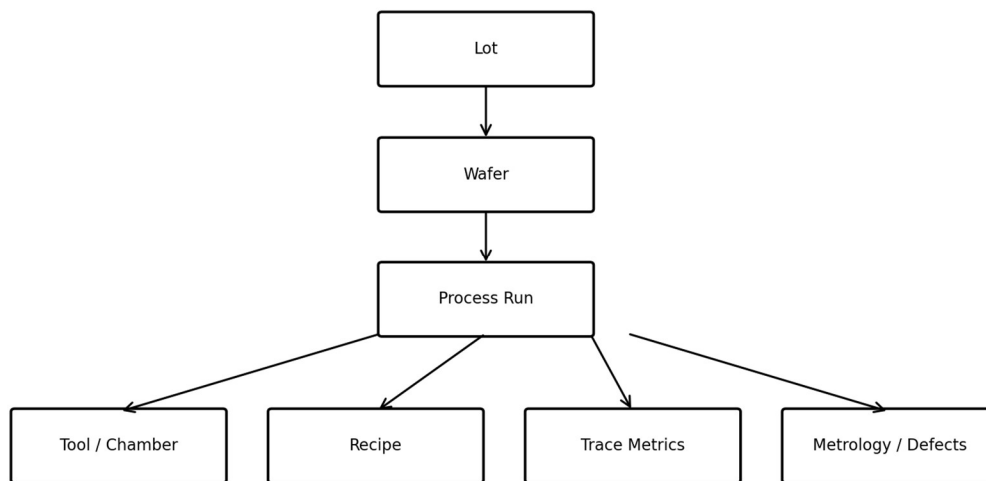
6. System Components

Component	Responsibility
-----------	----------------

Data ingestion	Accept structured manufacturing records and validate schemas
Validation and normalization	Check required fields, units, timestamps, identifiers, and ranges
Genealogy service	Connect lots, wafers, process runs, tools, chambers, and recipes
Feature engineering	Create historical, temporal, equipment, and spatial features
Anomaly service	Detect deviations from baseline or learned operating behavior
Defect pattern service	Analyze wafer coordinates and classify spatial distributions
Similar-lot service	Retrieve historically comparable lots and outcomes
Pre-run risk service	Score upcoming lots using only pre-run-safe information
Root-cause fusion service	Combine defined evidence signals into candidate scores
Recommendation service	Map evidence-supported candidates to reviewable next actions
Alert service	Expose risk and excursion events to the dashboard
Audit/action service	Record reviewer identity, decision, timestamp, and status

7. Data Model and Genealogy

Manufacturing Genealogy and Evidence Relationships



Genealogy is the foundation of traceable manufacturing analytics. Every analytical conclusion should be able to identify the lot, wafer, run, equipment, and time window from which its evidence was derived.

- Use stable identifiers for lots, wafers, tools, chambers, recipes, runs, defects, and evidence.
- Preserve event timestamps and distinguish event time from ingestion time.
- Avoid joining records only by approximate time when a stronger genealogy key exists.
- Record missingness, quality flags, and unit conversions.
- Prevent future information from entering a pre-run prediction record.

8. Algorithmic Design

8.1 Baseline and Anomaly Detection

A baseline describes expected behavior for a defined population, such as a chamber, recipe, product, or process step. A standardized deviation may be calculated as:

$$z = (x - \mu) / \sigma$$

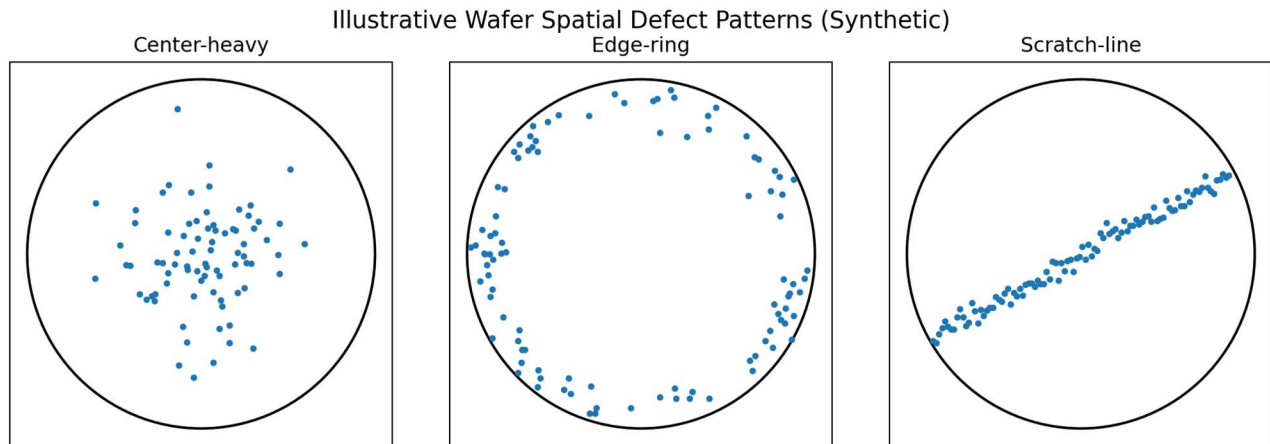
where x is the observation, μ is the baseline mean, and σ is the baseline standard deviation. Robust alternatives such as median/MAD or rolling baselines may be preferable when distributions contain outliers or drift.

- Z-score or robust deviation for univariate signals.
- Rolling statistics for time-dependent drift.
- EWMA or control-chart logic for gradual shifts.
- Isolation Forest or another multivariate method only when justified by data volume and validation.

Interpretation rule

An anomaly means that a value is unusual relative to a reference population. It does not, by itself, establish that the anomaly caused yield loss.

8.2 Spatial Wafer Defect Pattern Analysis



For a defect coordinate (x, y) and wafer center (cx, cy) , radial distance can be calculated as:

$$r = \sqrt{(x - cx)^2 + (y - cy)^2}$$

- Center-heavy: high concentration near the wafer center.
- Edge-ring: elevated concentration near the wafer boundary.
- Scratch-line: approximately linear or elongated defect distribution.
- Radial: distribution aligned with radial directions.
- Localized hotspot: concentrated cluster in a limited region.

Pattern labels are investigation features. Their physical interpretation depends on process context, inspection modality, wafer geometry, and defect taxonomy.

8.3 Similar-Lot Analysis

Similar-lot analysis retrieves historical records that resemble the target case using selected dimensions such as recipe, product, chamber, process window, maintenance state, and time proximity.

- Define similarity features before evaluating outcomes.
- Avoid selecting similar lots using post-outcome information that would not have been available at prediction time.
- Report the number of comparable lots and the similarity criteria.
- Show both supporting and contradictory examples when available.

8.4 Pre-Run Risk Prediction

Pre-run prediction must use only information known before the target lot starts. This is a temporal data-governance requirement, not merely a model setting.

Allowed feature examples	Disallowed feature examples
Planned recipe and route	Target lot's future yield
Assigned tool/chamber	Target lot's future defect map
Historical chamber yield	Target lot's post-run metrology
Maintenance completed before prediction time	Future maintenance events
Historical process distributions	Runtime anomaly values from the target lot

Candidate model families include logistic regression for interpretability, random forests or gradient boosting for nonlinear tabular patterns, and a transparent historical baseline. A risk score must not be called a calibrated probability unless calibration has been evaluated.

8.5 Seven-Signal Root-Cause Evidence Fusion

The repository describes deterministic fusion across seven evidence signals:

- Historical association
- Anomaly strength
- Defect-pattern compatibility
- Chamber/equipment recurrence
- Temporal precedence
- Maintenance evidence
- Similar-lot evidence

A general weighted formulation is:

$$S(c) = \sum w_i \times e_i(c)$$

where $S(c)$ is the candidate score, $e_i(c)$ is a normalized evidence signal, and w_i is the defined weight. The exact weights, normalization, missing-data treatment, and confidence mapping must be taken from the implementation and documented as a versioned configuration.

Governance rule

The root-cause service should produce a candidate ranking, not a claim of physical causation. Every candidate must include evidence IDs, `generated_at`, `algorithm_version`, and a confidence or uncertainty representation.

9. Scenario Walkthroughs

Etch drift

A chamber's pressure or power profile deviates from its historical baseline. LogicX detects the deviation, links it to affected runs, checks recurrence, compares similar lots, and presents a candidate process-drift hypothesis.

Overlay excursion

Metrology indicates an alignment deviation. The system connects the measurement to recipe, tool, time window, and comparable lots, while keeping the result as an investigation hypothesis.

Particle excursion

Inspection records show increased particle-like defects. Spatial distribution, chamber recurrence, maintenance timing, and similar-lot evidence are combined to prioritize investigation.

Cycle-time or temporal exposure

The system evaluates features such as time since maintenance, chamber usage count, process duration, or sequence position, provided these features are available before the relevant prediction point.

10. Frontend and User Experience

View	Expected purpose
Monitor	Fleet-level yield, excursion, and chamber recurrence overview
Investigate	Lot-level evidence, genealogy, anomaly timeline, and defect patterns
Predict	Pre-run risk for upcoming lots with feature provenance
Alerts	Prioritized risk and excursion notifications
Evidence	Evidence IDs, supporting records, timestamps, and algorithm versions
Actions	Recommendations, reviewer identity, approval status, and audit trail

The UI should never hard-code a root cause or a recommendation. It should render API responses, show the evidence behind the result, and make uncertainty visible.

11. Backend and Technology Stack

Layer	Technology / role
Frontend	React 18, TypeScript, Vite, Tailwind CSS
API	Python and FastAPI
ORM	SQLAlchemy
Analytics	pandas, NumPy, SciPy, scikit-learn
Database	PostgreSQL with SQLite fallback for lightweight environments
Migrations	Alembic
Testing	pytest and TypeScript compiler checks
Deployment	Docker and Docker Compose

The stack supports a modular prototype, but production readiness requires additional controls including authentication, authorization, observability, data contracts, reliability engineering, security review, and fab-specific validation.

12. IBM Bob Integration

IBM Bob is positioned as a development and SDLC agent, not as the runtime root-cause authority.

Bob workflow	Use in the project
Plan	Architecture decomposition, acceptance criteria, and

	implementation plan
Agent	Implement narrow backend, frontend, or test tasks
Ask	Diagnose development errors using repository context
Review	Review code quality, test coverage, and requirements
MCP / structured tools	Support controlled access to structured project or analytics information when configured

Judge-ready wording

We used IBM Bob to accelerate structured software development. The runtime prediction and root-cause pipeline remains a controlled analytics subsystem with explicit evidence, versioning, and human review.

13. Human-in-the-Loop and Safety

- No direct machine control or automatic recipe changes.
- Recommendations must be reviewed and approved by an authorized engineer.
- Every action should retain reviewer identity, timestamp, decision, and rationale.
- Low-confidence or contradictory evidence should be flagged for manual investigation.
- Data access should be role-based in any production-oriented deployment.
- Model and rule changes should be versioned and evaluated before release.

14. Validation and QA Plan

Test area	Validation objective
Schema validation	Reject missing, malformed, or invalid records
Genealogy integrity	Verify lot/wafer/run relationships and referential consistency
Temporal leakage	Prove pre-run features exclude future target-lot information
Anomaly correctness	Test known injected deviations and expected flags
Pattern classification	Evaluate synthetic pattern labels against known generators
Root-cause fusion	Check deterministic repeatability and score bounds
API contracts	Validate request/response schemas and error handling
Frontend typing	Run TypeScript compiler and component contract checks
Reproducibility	Regenerate seeded synthetic data and compare outputs
Security and governance	Check action approval and audit constraints

Any claim such as accuracy, recall, false-alert rate, time saved, or financial benefit should be supported by a reproducible evaluation protocol. Synthetic test performance must be clearly labeled as synthetic.

15. Live Demonstration Script

7. Open Monitor and explain the fleet-level context.
8. Select a synthetic affected lot with a known injected scenario.
9. Open Investigate and show genealogy, process traces, metrology, and inspection evidence.
10. Show the wafer map and explain the spatial pattern without overclaiming physical causation.
11. Open Evidence and show candidate causes, scores, evidence IDs, timestamps, and algorithm version.
12. Open Predict and select an upcoming lot; explain that only pre-run-safe features are used.
13. Open Actions and demonstrate that a recommendation requires reviewer approval.
14. Show the audit record and explain reproducibility and accountability.

16. Judge Questions and Answers

Q: Why is this not just an AI chatbot?

A: It is a structured analytics platform with data ingestion, genealogy, statistical analysis, defect pattern recognition, pre-run risk scoring, evidence fusion, APIs, and audit workflows. Bob supports development; it is not the runtime prediction engine.

Q: Can the system prove the root cause?

A: No. It ranks evidence-supported candidate causes. Physical causation requires engineering validation, controlled experiments, and domain-specific confirmation.

Q: How do you prevent leakage?

A: We define a pre-run feature whitelist and audit inputs so the model cannot use future yield, defects, metrology, or maintenance events for the target lot.

Q: Why use synthetic data?

A: Fab data is proprietary and sensitive. Deterministic synthetic data supports reproducible demonstrations without claiming access to a real factory.

Q: What happens if the ranking is wrong?

A: The system exposes evidence and uncertainty, requires engineer review, and does not directly control manufacturing equipment.

Q: How will this scale?

A: The architecture separates ingestion, storage, analytics services, APIs, and UI. Scaling requires production data contracts, workload testing, observability, security, and validation.

Q: What is the measurable benefit?

A: The prototype's defensible immediate benefit is structured evidence aggregation and prioritization. Quantitative time or financial benefits must be measured in a controlled benchmark.

17. Limitations and Honest Claims

- The repository uses synthetic deterministic demonstration data; it does not represent an operating fab.
- Historical association does not establish causation.
- Risk scores are not automatically calibrated probabilities.
- Spatial defect patterns are contextual signals, not universal diagnoses.
- Production deployment would require fab-specific integration and validation.
- Financial impact claims require verified manufacturing and economic data.
- Model drift, changing recipes, new products, and equipment changes can affect performance.

18. Future Roadmap

Phase	Potential enhancement
Phase 1	Complete reproducible synthetic scenarios, API contracts, and automated QA
Phase 2	Add stronger baseline management, drift monitoring, and model calibration

Phase 3	Add role-based access, audit export, and operational observability
Phase 4	Integrate representative anonymized or partner-approved datasets
Phase 5	Validate with process engineers and compare against current investigation workflow
Phase 6	Introduce controlled experiment feedback and continuous model governance

19. Conclusion

LogicX addresses a meaningful semiconductor manufacturing problem by connecting fragmented manufacturing evidence into a structured investigation and pre-run risk workflow. Its strongest technical message is not that AI can magically identify a cause; it is that a controlled analytics system can organize evidence, preserve temporal correctness, make candidate explanations auditable, and keep engineers in control.

Final judge message

LogicX converts fragmented wafer manufacturing data into an evidence-driven engineering workflow: detect the deviation, understand the pattern, connect the genealogy, rank candidate causes, assess upcoming-lot risk, and support a traceable engineer decision.

20. Source and Data Disclaimer

Repository reference: <https://github.com/ronaksavsani181/bob-ai-hackathon-LogicX>

The public repository documentation describes the architecture, technology stack, synthetic data approach, guardrails, and intended workflows. Exact implementation behavior, test results, algorithm weights, and runtime performance should be confirmed from the checked-out source and executed test suite before being stated as verified facts in a judging presentation.

All manufacturing examples in this document are illustrative. They are not operational measurements from a named semiconductor fab and should not be represented as proprietary or production-validated data.